

EVO-HARNCESS: 面向自进化智能体的上下文到技能框架编译

Tianxin Wei¹, Zhan Shi², Minhua Lin³, Bing He², Zewen Liu⁴, Yisi Sang²,
Yuanchen Bei¹, Xuying Ning¹, Jiaru Zou¹, Ting-Wei Li¹, Xiao Lin¹, Yanjun Zhao¹,
Chi Wang⁵, Benoit Dumoulin², Dakuo Wang⁶, Jingrui He¹, Hanqing Lu²

¹University of Illinois Urbana-Champaign ²Amazon ³The Pennsylvania State University
⁴Emory University ⁵AG2 AI ⁶Northeastern University

摘要

从经验中学习对于开发能力强、能够自我改进的大语言模型（LLM）智能体至关重要。现有方法通常通过反思、记忆、规则或技能从累积的轨迹中提取知识。然而，现实环境中的智能体会不断遇到新任务，往往只有一次改进机会。这些执行过程产生丰富但高度嘈杂的上下文，将广泛有用的经验与任务特定的工件纠缠在一起。关键的是，先前的工作很少在复杂的现实世界任务上验证其有效性，或分离出改进的潜在驱动因素。为了解决这些不足，我们提出了在线框架学习（online harness learning），其中冻结的智能体通过跨顺序任务持续更新结构化框架来实现改进。该公式使我们能够通过所提出的 Evo-Harness 系统地研究关键的自我改进因素。其核心是上下文到技能框架编译，将嘈杂的单个执行提炼为可复用的技能框架，以实现跨领域和主题级别的适应。为了证明单次技能编译的有效性，我们在五个现实基准（TerminalBench2、SWE-bench、CL-Bench、 τ bench、WebArena-Infinity）上进行了评估。我们广泛的分析证明了 EVO-HARNCESS 的有效性，并为大语言模型智能体如何有效地即时学习提供了原则性的理解。我们的代码可在 [Link](#) 获取。

1 引言

大语言模型（LLM）智能体（Achiam et al., 2023; Team et al., 2023; Liu et al., 2024）在复杂的任务解决场景中表现出强大的性能，例如网络交互（Zhou et al., 2024）、软件工程（Jimenez et al., 2024）、工具使用（Qin et al., 2024）和长程推理（Erdogan et al., 2025; Sun et al., 2025）。这些场景要求智能体进行规划、行动、解释观察、处理反馈并从错误中恢复。然而，当智能体在任务中失败时，

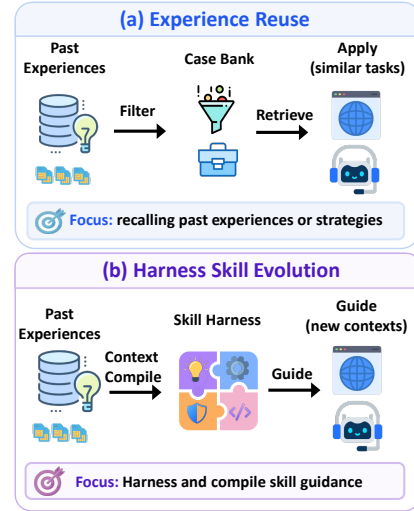


图 1: 经验复用（检索过往案例）与技能框架演化（将经验编译为可复用指导）的比较

失败往往只是一个孤立事件，而非未来改进的源泉。若不将失败的执行转化为可复用的经验，智能体在后续任务中会继续犯类似的错误。许多基于经验的方法（Shinn 等，2023；Zhao 等，2024）通过离线地从累积轨迹中学习（Lee 等，2026；Ni 等，2026）来解决这一问题，利用反思（Shinn 等，2023；Qiu 等，2026）、记忆（Zhong 等，2024；Chhikara 等，2025；Xu 等，2025）和技能（Zhang 等，2026a；Xia 等，2026；Jiang 等，2026；Yang 等，2026）等媒介。然而，在现实场景中，智能体不断遇到新任务和新情境，每次完成的执行通常只提供一次改进未来行为的机会。此外，真实执行会产生丰富但高度嘈杂的情境，将有用的经验与任务特定细节、部分失败和偶然的工具痕迹纠缠在一起。关键的是，先前的方法往往未能在这些现实约束下严格验证其有效性，也未能系统地分离出智能体改进的潜在驱动因素。为了导航

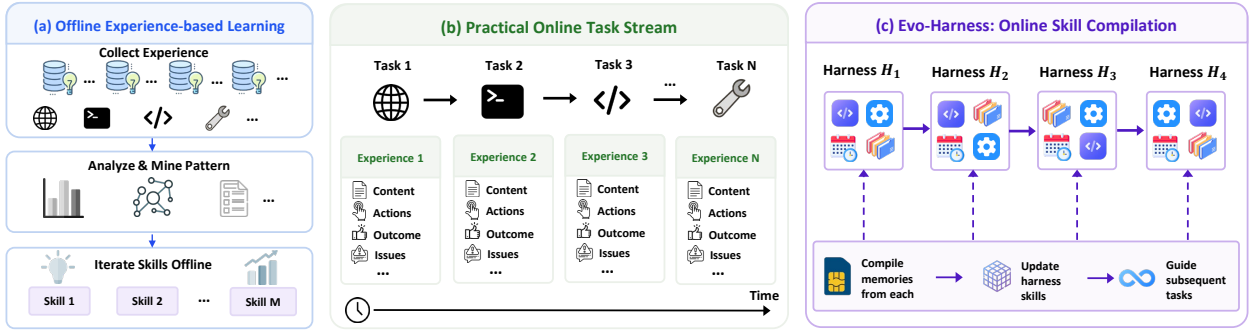


图 2: 问题设定示意图。(a) 先前方法收集执行并离线挖掘可复用模式。(b) 在实际的在线流中, 智能体顺序遇到任务, 产生丰富但一次性的执行经验。(c) EVO-HARNES 将这些在线经验编译为不断演进的技能束, 以供未来复用。

在如此一次性、复杂的学习环境中, 仅依赖对过往经验的简单检索是不够的。相反, 智能体需要主动将原始执行上下文编译为可复用的技能指导, 以应对新任务, 如图 1 所示。基于这一观点, 我们提出了在线驾驭学习 (online harness learning), 其中冻结的智能体通过在连续任务中逐步更新一个结构化的外部驾驭器来实现改进。为了应对不断涌现的新任务, 该驾驭器充当一个自我进化的学习媒介, 逐步将执行过程提炼为面向未来的可操作指导。与标准离线学习相比, 这一实用设置的概览如图 2 所示。为了具体化这一表述并支持对持续自我改进的系统研究, 我们引入了进化驾驭器 (Evo-Harness)。在该框架内, 我们使用技能来实例化驾驭器。技能作为现实任务执行的自然媒介, 因为它们能够将多样化的学习信号 (如失败、反馈、约束和操作流程) 整合为结构化知识, 从而塑造未来的规划与恢复。进化驾驭器的核心是上下文到驾驭器的技能编译, 它将单个执行上下文转换为可复用的技能驾驭器更新。具体而言, 我们的编译过程使用求解器对上下文进行反思并提出候选技能, 同时使用进化器更新现有驾驭器。通用技能捕捉可迁移的跨任务模式, 而主题技能则捕捉局部化的操作知识。这种双重设计在保留关键学习信号的同时, 过滤了轨迹级别的噪声。我们在五个现实的基准 (终端基准 2、软件工程基准、持续学习基准、 τ 基准和无限网络竞技场) 上, 以顺序持续学习设置评估了进化驾驭器。除了展示性能提升外, 我们的主要关注点在于

本文系统性地探究了自我提升的核心驱动因素。通过分析进化器设计、反馈类型与迁移设置如何决定性能, 本研究将技能利用机制视为一个可解释对象, 用以理解冻结的大语言模型智能体如何即时学习。本文的贡献总结如下:

- **在线线束学习。**本文提出一种设定: 冻结的智能体通过增量更新可复用的外部框架而非模型参数, 在顺序任务流上实现性能提升。
- **上下文到驾驭技能编译。**本文引入一种机制, 将单次执行上下文编译为可操作的一般性和主题级技能更新。
- **系统分析框架演化。**本文分离了演化器设计、反馈类型和迁移设置对持续自我改进的影响。在五个复杂基准上评估, EVO-HARNES 始终优于现有的基于经验的方法。

2 相关工作

在本节中, 我们回顾了关于自进化大语言模型智能体以及用于智能体经验的外部记忆或技能库的现有工作。

进化与自我改进的大语言模型智能体。一系列不断增长的研究探索了大型语言模型智能体如何通过交互和反馈实现自我改进。除了基于标量或语言反馈迭代修改输出的局部反思方法 (Shinn 等, 2023; Madaan 等, 2023), 基于经验的方法尝试离线提取可复用的知识。例如, 一些工作从累积的轨迹池中挖掘, 以合成自然语言洞见 (Zhao 等, 2024)、交互规则 (Chen 等, 2024) 或可迁移的指导。

ries (Ni 等, 2026)。类似地, 另一条并行研究路线聚焦于被动演化更高层级的智能体工件, 例如提示 (Suzgun 等, 2025; Lou 等, 2026)、推理库 (Ouyang 等, 2025) 或记忆结构 (Agrawal 等, 2025; Lee 等, 2026; Zhang 等, 2025; Wei 等, 2025; Lin 等, 2026a)。为了提供更具操作性的指导, 近期方法已转向以技能为中心的演化。这包括构建可执行代码技能 (Wang 等, 2023)、通过策略学习优化技能库 (Xia 等, 2026; Zhang 等, 2026a; Ouyang 等, 2026), 或整理技能知识库与表示 (Yang 等, 2026; Qiu 等, 2026; Jiang 等, 2026; Wang 等, 2026)。尽管密切相关, 这些现有框架大多将学习到的工件视为用于自由形式检索的被动存储库。关键的是, 先前方法往往未能在现实的在线约束下严格验证其有效性, 也未能系统性地分离出智能体改进的潜在驱动因素。我们的工作通过将单次经验编译为主动技能框架来弥补这一空白, 提供了一个结构性支架, 以系统研究大语言模型智能体如何即时学习。

记忆与技能驾驭框架用于智能体经验。记忆为保存智能体经验提供了基础的外部媒介。现有记忆系统维护长期观察历史 (Zhong 等, 2024; Yu 等, 2025), 通过自适应结构组织记忆 (Xu 等, 2025), 并探索跨模态更新与检索证据的方法 (Huang 等, 2026; Bei 等, 2026; Liu 等, 2026a)。然而, 这些方法大多将记忆视为被动的信息存储。技能则根本不同, 因为它们编码操作性知识。它们从仅仅记录发生了什么, 转变为通过手册明确规定智能体应如何行动 (Chen 等, 2024), 通过有组织的目录验证操作 (Ni 等, 2026), 或利用技能库从失败中恢复 (Ouyang 等, 2026; Wang 等, 2026; Zhang 等, 2026b)。这种程序化视角将技能提升为一种主动的驾驭框架。驾驭框架不仅提供历史上下文, 还充当外部脚手架, 从结构上塑造未来的规划, 其形式包括模式库 (Ning 等, 2026; Qiu 等, 2026)、推理库 (Ouyang 等, 2025) 和动态指导文件 (Suzgun 等, 2025; Lee 等, 2026; Liu 等, 2026b; Lin 等, 2026b)。

基于这一区别, 我们的工作将技能驾驭框架概念化为不仅是一种提升性能的工件, 更是一种可解释的媒介, 用于系统研究持续的在线适应。为了应对真实世界执行中的高噪声, EVO-HARNES 将单次反思编译为包含通用技能和主题技能的结构化驾驭框架。虽然这种分离能有效过滤任务特定的工件, 但其在我们工作中的主要目的是提供一个透明的脚手架。这明确允许我们系统地隔离和评估核心驱动因素, 例如进化器设计和反馈类型, 这些因素决定了冻结的大语言模型智能体如何即时自我改进。

3 EVO-HARNES: 上下文到驾驭框架技能编译

我们提出 EVO-HARNES, 一个用于在线驾驭框架学习的框架。其目标是将一次性执行上下文转换为可复用的驾驭框架, 以指导未来的任务解决。与回忆过去案例的经验检索不同, EVO-HARNES 将嘈杂的执行信号编译为结构化指导, 从而塑造后续的规划、行动、验证和恢复。

图 3 给出了概览。进化式引导框架 (EVO-HARNES) 在执行前维护一个当前引导框架, 用于指导求解器, 塑造其规划、行动、验证和恢复的方式。在每个任务批次之后, 执行上下文被反思为候选记忆, 并编译回引导框架中, 形成跨任务模式和任务类型流程, 使更新后的引导框架能够指导后续执行。

3.1 在线引导框架学习设置

设 $\{B_1, \dots, B_K\}$ 表示任务批次流, 其中 $B_i = \{x_{i,1}, \dots, x_{i,m_i}\}$ 。在批次 B_i 之前, 智能体拥有一个当前引导框架 $\mathcal{H}_i$, 这是一组结构化的可复用指导条目:

$$\mathcal{H}_i = \{h_i^1, \dots, h_i^{n_i}\}. \quad (1)$$

引导框架位于冻结求解器外部, 作为后续执行的控制脚手架。与原始轨迹存储不同, 它维护从先前执行中提炼出的紧凑指导, 并通过选择并注入求解器上下文来影响未来行为。对于每个任务 $x_{i,j}$, 智能体选择一组紧凑的引导框架条目:

$$S_{i,j} = \text{Select}(x_{i,j}, \mathcal{H}_i; b), \quad |S_{i,j}| \leq b, \quad (2)$$

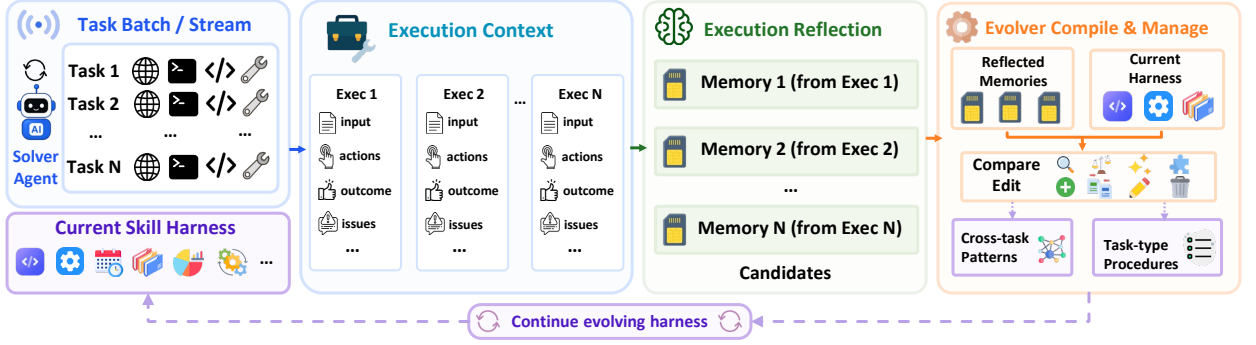


图 3: 进化式引导框架 (EVO-HARNES) 概览。求解器使用当前引导框架来解决任务批次或流。每次执行产生一个上下文，包括输入、动作轨迹、结果和反馈。执行反思将这些上下文转换为候选记忆。进化器将候选记忆与当前引导框架进行比较，并用跨任务模式和任务类型流程更新引导框架，以供后续批次使用。

其中 b 是注入预算。所选指导被注入到任务中：

$$\tilde{x}_{i,j} = \text{Inject}(x_{i,j}, \mathcal{S}_{i,j}). \quad (3)$$

冻结求解器 $\mathcal{A}$ 执行任务：

$$(\tau_{i,j}, y_{i,j}, f_{i,j}) = \mathcal{A}(\tilde{x}_{i,j}), \quad (4)$$

其中 $\tau_{i,j}$ 是动作轨迹， $y_{i,j}$ 是结果， $f_{i,j}$ 是可用的反馈，例如验证器结果、单元测试输出、工具诊断或评判反馈。执行上下文为：

$$c_{i,j} = (x_{i,j}, \tau_{i,j}, y_{i,j}, f_{i,j}). \quad (5)$$

在线束学习更新 $\mathcal{H}_i$ 至 $\mathcal{H}_{i+1}$ ，源自这些执行上下文，同时保持求解器参数固定。

3.2 上下文到束的编译

EVO-HARNES 采用两个互补阶段：执行反思与束进化。反思从每次执行中提取学习信号。进化将这些信号整合到束中。

执行反思。对于每个已完成的执行上下文 $c_{i,j}$ ，求解器仅在执行失败或收到负面反馈时生成候选记忆：

$$r_{i,j} = \text{Reflect}(c_{i,j}). \quad (6)$$

每个候选记忆表示为：

$$r = (\text{教训}, \text{触发条件}, \text{证据}, \text{作用域_提示}). \quad (7)$$

反思利用执行后立即可用的完整任务侧上下文，包括指令、轨迹、观察结果、工具输出，

结果与反馈。本文将反思聚焦于失败案例，因为这些案例暴露了求解器当前的边界：错误的假设、缺失的约束、无效的工具使用、薄弱的验证以及失败的恢复。这使得生成的候选记忆更具针对性，有助于改进引导机制，同时避免从可能包含任务特定细节的成功执行中进行不必要的更新。反思阶段不与当前引导机制进行比较，因此其输出并非最终的引导机制更新，而是从完整执行中提取的、以失败为基础的候选信号。

引导机制演化。在批次 B_i 之后，候选 memories form:

$$\mathcal{R}_i = \{r_{i,j}\}_{j=1}^{m_i}. \quad (8)$$

演化器接收候选记忆和当前引导机制：

$$\mathcal{O}_i = \text{Evolver}(\mathcal{H}_i, \mathcal{R}_i), \quad (9)$$

其中 $\mathcal{O}_i$ 是一组引导机制编辑。从概念上讲，演化器应用：

$$o = \pi_{\text{evolve}}(r, \mathcal{H}_i, \mathcal{R}_i), \quad (10)$$

where $o \in \{\text{ADD}, \text{MERGE}, \text{REVISE}, \text{SKIP}\}$. The 演化器过滤噪声或冗余记忆，合并兼容信号，修订现有指导，并在适当情况下提升可复用经验。更新后的引导机制为：

$$\mathcal{H}_{i+1} = \text{ApplyEdits}(\mathcal{H}_i, \mathcal{O}_i). \quad (11)$$

这使得引导机制不仅仅是一个记忆存储：候选记忆并非直接追加，而是被编译为能够影响未来行为的指导。

Algorithm 1 EVO-HARNESS: Context-to-Harness Skill Compilation

```
1: Input: task batches  $\{B_1, \dots, B_K\}$ , frozen solver  $\mathcal{A}$ ,  
   initial harness  $\mathcal{H}_1$   
2: for  $i = 1, \dots, K$  do  
3:    $\mathcal{R}_i \leftarrow \emptyset$   
4:   for each task  $x_{i,j} \in B_i$  do  
5:      $\mathcal{S}_{i,j} \leftarrow \text{Select}(x_{i,j}, \mathcal{H}_i; b)$   
6:      $\tilde{x}_{i,j} \leftarrow \text{Inject}(x_{i,j}, \mathcal{S}_{i,j})$   
7:      $(\tau_{i,j}, y_{i,j}, f_{i,j}) \leftarrow \mathcal{A}(\tilde{x}_{i,j})$   
8:      $c_{i,j} \leftarrow (x_{i,j}, \tau_{i,j}, y_{i,j}, f_{i,j})$   
9:      $r_{i,j} \leftarrow \text{Reflect}(c_{i,j})$   
10:     $\mathcal{R}_i \leftarrow \mathcal{R}_i \cup \{r_{i,j}\}$   
11:  end for  
12:   $\mathcal{O}_i^{\text{type}} \leftarrow \text{CompileTaskType}(\mathcal{H}_i, \mathcal{R}_i)$   
13:   $\mathcal{O}_i^{\text{cross}} \leftarrow \text{CompileCrossTask}(\mathcal{H}_i, \mathcal{R}_i)$   
14:   $\mathcal{O}_i \leftarrow \mathcal{O}_i^{\text{type}} \cup \mathcal{O}_i^{\text{cross}}$   
15:   $\mathcal{H}_{i+1} \leftarrow \text{ApplyEdits}(\mathcal{H}_i, \mathcal{O}_i)$   
16: end for  
17: Output: evolved harness  $\mathcal{H}_{K+1}$  and task results
```

3.3 引导机制层级

演化后的框架包含两个层级的可复用指导。跨任务模式通过比较不同任务间的记忆并识别共享的操作结构来编译。任务类型程序则为重复出现的任务格式、接口或领域保留更局部的指导。这些层级并非针对单次执行的预定义标签。它们源于演化器如何筛选、比较并将候选记忆整合进框架。

3.4 算法与模块化

Algorithm 1 summarizes EVO-HARNESS. We use batch-level updates so the evolver can both compile localized task-type procedures and compare memories across tasks to extract cross-task patterns. The same formulation reduces to task-type updating when $|B_i| = 1$.

Here, `CompileTaskType` preserves localized procedures, while `CompileCrossTask` extracts shared patterns across executions. The modular design is intentional: by decoupling the solver, evolver, feedback source, and initial harness state, EVO-HARNESS provides a controlled framework for systematically studying what drives online harness improvement.

4 实验

我们进行实验以评估 EVO-HARNESS，并理解在线框架演化何时能改进 LLM 智能体：

- 研究问题 1：整体有效性。EVO-HARNESS 是否能在多样任务中改进冻结的智能体

benchmarks, and which task categories benefit most?

- 研究问题 2：框架内容与组件。演化后的框架产生了哪些类型的指导，不同框架组件如何贡献？
- 研究问题 3：在线更新与迁移。演化后的框架在训练集划分迁移、在线更新以及求解器-演化器配对下表现如何？
- 研究问题 4：反馈基础。反馈的来源和粒度如何影响框架的演化？

4.1 设置

实验设置。我们在五个基准上评估了 EVO-HARNESS，这些基准涵盖了多样化的智能体能力：用于网页导航的 WebArena-Infinity (Zhou, 2026)、用于命令行任务的 TerminalBench2 (Merrill et al., 2026)、用于软件工程的 SWE-bench Lite (Jimenez et al., 2024)、用于自适应推理的 CL-Bench (Dou et al., 2026) 以及用于工具使用任务的 TAU-Bench (Yao et al., 2024)。我们使用 Claude Opus 4.6 (Anthropic, 2026a) 作为默认求解器和演化器，并在研究模型消融时额外评估了 Claude Opus 4.7 (Anthropic, 2026b)、Claude Opus 4.5 (Anthropic, 2025)、Kimi-K2.5 (Team et al., 2026) 和 GPT-OSS (Agarwal et al., 2025)。我们以百分比报告成功率，数值越高越好。更多基准和实现细节见附录 A 和 F。

基线。我们在相同的任务划分和评估协议下，与具有代表性的基于经验、记忆和技能的适应方法进行了比较。AWM (Wang et al., 2024) 重用智能体工作记忆，Dynamic Cheatsheet (Suzgun et al., 2025) 维护一个自适应的外部备忘录，Evo-Memory (Wei et al., 2025) 从执行中演化可重用记忆，ACE (Zhang et al., 2025) 优化智能体上下文，XSkill (Jiang et al., 2026) 将经验和技能分离以实现持续重用。DC-Cu 和 DC-RS 是 Dynamic Cheatsheet 的两个变体。

4.2 总体有效性 (RQ1)

表 1 报告了以 Claude Opus 4.6 作为求解器的主要结果。**EVO-HARNESS** 相较于 **No-Evolve** 和先前基于经验的方法均有提升

在所有五个基准上的基线。增益在终端基准 2 上尤为显著，其中智能体需要检查文件、执行命令、解释错误并从失败操作中恢复。这表明当任务暴露可复用的程序性结构时，框架进化尤为有用。与最强基线 XSkill 相比，进化框架仍在每个基准上有所提升。外部经验复用并非自动有益。

表 1：使用克劳德·欧普斯 4.6 作为求解器的主要结果。分数表示成功率（%），数值越高表示性能越好。进化框架在所有五个基准上均取得最佳结果。

Method	CL-Bench	TerminalBench-2	SWE-bench Lite	τ -bench	WebArena-Infinity
No Evolve	29.54	62.92	63.67	72.73	72.50
AWM	28.91	62.92	62.67	70.91	70.00
DC-Cu	29.23	60.67	63.00	72.12	71.25
DC-RS	29.07	61.80	61.67	70.91	70.00
Evo-Memory	29.38	64.04	64.00	72.12	72.50
ACE	29.70	61.80	63.67	72.73	71.25
XSkill	31.44	66.29	64.67	73.94	73.75
EVO-HARNESS	34.02	73.03	67.00	76.97	76.25

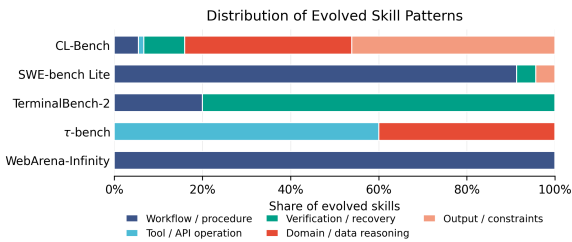


图 4：跨基准的进化技能模式分布。所学技能展现出基准特定的特征，涵盖工作流与工具使用流程、验证、领域推理以及输出约束处理。

解释错误并从失败操作中恢复。这表明当任务暴露可复用的程序性结构时，框架进化尤为有用。与最强基线 XSkill 相比，进化框架仍在每个基准上有所提升。外部经验复用并非自动有益。

有益。若干基线在某些基准上表现不及无进化，表明检索或进化的指导可能具有噪声、过于具体或与求解器不匹配。这支持了我们的动机：不仅研究自我进化是否有帮助，还研究哪些因素使在线框架更新有效。

4.3 类别与模型通用性（研究问题 1）

表 2 提供了 **CL-Bench** 在五个求解器模型上的代表性类别级视图。由于 **CL-Bench** 涵盖多样化的推理与执行类别，它使我们能够考察框架演化在何处最为有效，以及其效果如何随求解器能力而变化。**EVO-HARNESS** 提升了每个求解器模型的总体得分，且在更强的 **Claude** 模型上增益更大

相较于开放权重替代方案。Opus 4.7、Opus 4.6 和 Opus 4.5 分别获得 +3.7, +4.5 、

和 +3.8 的增益，而 Kimi-K2.5 和 GPT-OSS 分别提升 +1.1 和 +0.8 。这表明更强的求解器可能更善于解读、遵循并受益于演化后的框架指导。在各类别中，最一致的改进出现在程序性任务执行（PTE）上，每个模型都有所提升，且 Opus 4.6 和 Opus 4.5 各获得 +9.8 分。当任务暴露可复用的

操作结构时，框架演化最为可靠。相比之下，经验发现与仿真（EDS）的增益较不稳定，Opus 4.6 和 GPT-OSS 出现下降。这表明更具探索性的任务可能对过于具体的指导更为敏感。

4.4 框架内容与组件（RQ2）

图 4 分析了跨基准的演化框架内容。演化后的框架反映了基准特定的操作需求，而非累积通用指导

ance. WebArena-Infinity 主要产生工作流和过程技能，TerminalBench-2 强调验证和恢复， τ -bench 混合了工具/API 操作与领域推理，而 CL-Bench 产生更广泛的分布，包括领域推理和输出约束处理。这些模式表明，上下文到框架的编译使框架适应每个环境的操作结构和失败模式。表 3 研究了框架更新过程的哪些部分对性能有贡献。我们将此分析及后续诊断分析主要聚焦于 CL-Bench 和 SWE-bench Lite，因为它们强调不同形式的框架指导：CL-Bench 包含异构自适应推理任务，其中局部任务类型指导可能很重要，而 SWE-bench Lite 强调仓库级调试和验证过程，预期更广泛的可重用指导会有所帮助。NO

表 2: 不同模型在 CL-Bench 类别上的结果: 五个模型的通过率 (%)。粗体 = 为每个类别-模型对的最佳结果。 Δ shows 为 EVO-HARNNESS 相对于 No-Evolve 基线的绝对改进。DKR = 领域知识推理, EDS = 经验发现与仿真, PTE = 过程任务执行, RSA = 规则系统应用。

Model	Method	DKR		EDS		PTE		RSA		All	
		Rate	Δ	Rate	Δ	Rate	Δ	Rate	Δ	Rate	Δ
Opus 4.7	No-Evolve	32.4	—	19.1	—	35.5	—	29.0	—	30.8	—
	EVO-HARNNESS	34.1	(+1.7)	19.6	(+0.5)	42.7	(+7.2)	32.7	(+3.7)	34.5	(+3.7)
Opus 4.6	No-Evolve	31.4	—	20.6	—	34.0	—	26.9	—	29.5	—
	EVO-HARNNESS	35.6	(+4.2)	18.1	(-2.5)	43.8	(+9.8)	30.4	(+3.5)	34.0	(+4.5)
Opus 4.5	No-Evolve	31.4	—	18.1	—	27.8	—	25.3	—	27.3	—
	EVO-HARNNESS	32.6	(+1.2)	19.1	(+1.0)	37.6	(+9.8)	28.3	(+3.0)	31.1	(+3.8)
Kimi-K2.5	No-Evolve	25.6	—	12.1	—	25.1	—	21.6	—	22.9	—
	EVO-HARNNESS	25.9	(+0.3)	13.1	(+1.0)	29.3	(+4.2)	20.8	(-0.7)	23.9	(+1.1)
GPT-OSS	No-Evolve	17.6	—	11.6	—	17.6	—	16.4	—	16.6	—
	EVO-HARNNESS	17.9	(+0.3)	10.6	(-1.0)	18.5	(+0.8)	18.6	(+2.1)	17.5	(+0.8)

表 3: 使用 Claude Opus 4.6 作为求解器在 CL-Bench 和 SWE-bench Lite 上的消融实验。分数表示通过率 (%)。

Method	CL-Bench	SWE-bench Lite
No Evolve	29.54	63.67
No Propose	33.28	65.33
General Only	30.28	66.67
Topic Only	33.70	64.33
EVO-HARNNESS	34.02	67.00

PROPOSE 移除了求解器侧对候选记忆的提议, GENERAL ONLY 仅保留跨任务指导, TOPIC ONLY 仅保留局部任务类型指导。广泛的跨任务指导和局部任务类型指导都有用, 但其重要性取决于任务设置。完整的 EVO-HARNNESS 设置在两个基准上均表现最佳。在 CL-Bench 上, Topic Only 达到 33.70, 接近完整结果的 34.02, 而 General Only 降至 30.28, 表明局部任务类型指导对其异构推理任务更有用。在 SWE-bench Lite 上, General Only 达到 66.67, 接近完整结果的 67.00, 而 Topic Only 达到 64.33, 表明广泛的调试、验证和仓库检查过程更重要。No Propose 优于 No Evolve 但仍低于完整方法, 表明提议和框架级编译是互补的。

4.5 迁移与适应性 (RQ3)

图 5 比较了三种技能更新设置。NO-EVOLVE 不使用框架更新。TRAIN-SPLIT

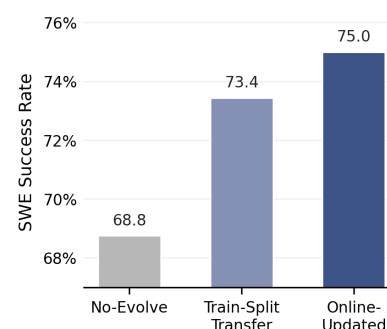


图 5: 不同演化设置下的 SWE-bench Lite 成功率。无演化 (NO-EVOLVE) 不使用测试框架更新; 训练集迁移 (TRAIN-SPLIT TRANSFER) 评估由 Claude Sonnet 在训练集上演化出的技能; 在线更新 (ONLINE-UPDATED) 在任务流中更新技能。

迁移 (TRANSFER) 使用由 Claude Sonnet 4.5 在训练集上演化出的技能, 并在测试集上用 Opus 4.7 求解器进行评估。在线更新

(ONLINE-UPDATED) 在测试任务流中直接用相同模型更新测试框架。在训练集上学习到的技能可以跨任务和模型迁移, 而在线更新

表现最佳。 无演化达到 68.8, 训练集迁移达到 73.4, 在线更新达到 75.0。训练集迁移带来的提升表明, 由较小或不同模型演化出的技能仍能为更强的求解器在未见任务上提供有用指导。在线更新比训练集迁移进一步提高了 +1.6 个百分点, 表明原位测试框架更新能更好地适应测试流的局部分布和失败模式。图 6 考察了求解器-演化器配对如何影响测试框架演化。我们比较两种求解器

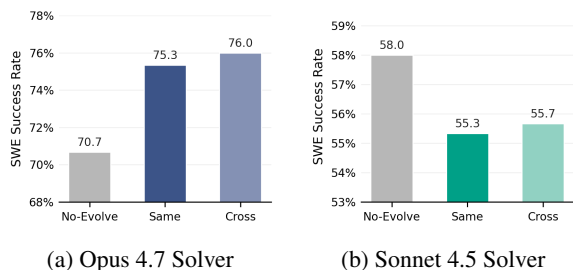


图 6: 求解器-演化器配对对 SWE-bench Lite 的影响。同源 (SAME) 表示使用同一模型进行求解和技能演化, 交叉 (CROSS) 表示求解器和演化器使用不同模型。

设置: Opus 4.7 求解器和 Sonnet 4.5 求解器。在每种设置中, 同源 (SAME) 使用同一模型同时作为求解器和演化器, 而交叉 (CROSS) 使用另一个模型作为演化器。具体而言, 对于 Opus 求解器, 同源使用 Opus 演化测试框架, 交叉使用 Sonnet 演化测试框架; 对于 Sonnet 求解器, 同源使用 Sonnet 作为演化器, 交叉使用 Opus 作为演化器。使用同一模型进行求解和演化并不总是最优的; 跨模型演化可能有所帮助, 但前提是求解器能够利用

演化出的指导。 对于 Opus 求解器, Cross 达到 76.0, 略高于 Same 的 75.3, 并明显高于 No-Evolve 的 70.7。这表明, 当下游求解器具备足够的解释和应用能力时, 不同的进化器可以产生仍然有用甚至更有用的指导。然而, 对于 Sonnet 求解器, Same 和 Cross 分别达到 55.3 和 55.7, 均低于 No-Evolve 的 58.0。因此, Sonnet 进化的技能可以帮助 Opus, 但不一定帮助 Sonnet 本身。这种对比表明, harness 迁移不仅取决于技能工件, 还取决于基础求解器的能力: 如果求解器不能可靠地遵循、情境化或调整指导, 进化可能无法提升性能。

4.6 反馈接地 (RQ4)

表 4 评估了反馈信号如何影响 CL-Bench 和 SWE-bench Lite 上的框架演化。本文比较了三种反馈设置。自生成 (SELF-GENERATED) 要求大语言模型自行判断执行是否成功以及应学习的内容。最小化 (MINIMAL) 仅提供环境层面的成功或失败信号, 例如命令、验证器或测试是否通过。标准 (STANDARD) 提供有依据的诊断反馈, 包括失败消息、错误迹、验证器输出或可用的测试失败信息。

表 4: 反馈级别对 CL-Bench 和 SWE-bench Lite 的影响, 使用 Claude Opus 4.6 作为求解器。分数表示通过率 (%)。

Feedback Level	CL-Bench	SWE-bench Lite
No Evolve	29.54	63.67
Self-Generated	27.96	61.67
Minimal	29.86	67.33
Standard	34.02	67.00

可靠的接地反馈至关重要

驾驭进化。 自生成反馈在两个基准上均不及不进化, 使 CL-Bench 从 29.54 降至 27.96, SWEbench Lite 从 63.67 降至 61.67。这表明, 当大语言模型生成的自判断缺乏外部执行证据支撑时, 可能引入误导性更新。相比之下, 最小反馈和标准反馈均利用环境提供的信号, 整体上更为可靠。标准反馈通常更优, 但反馈的有效粒度取决于

在基准测试上。 在 CL-Bench 上, 标准反馈表现最佳, 达到 34.02, 而最小反馈仅达到 29.86。这表明更丰富的诊断信息对于自适应推理任务是有用的, 因为理解失败的原因有助于编译出更好的引导框架。然而, 在 SWE-bench Lite 上, 最小反馈达到 67.33, 略高于标准反馈的 67.00。一种可能的解释是, 详细的错误信息有时会使演化后的引导框架过度依赖于特定任务的失败模式, 而稀疏的通过/失败信号则鼓励生成更具泛化性的引导。

5 结论

本文将在线引导框架学习形式化为一种研究设定, 用于探究冻结的大语言模型智能体如何从顺序的一次性执行中改进。通过 EVO-HARNESS, 我们以上下文到引导框架的技能编译来实例化这一设定, 将嘈杂的执行上下文转化为可复用的外部引导框架, 用于未来的任务求解。除了提出一个流程之外, 我们的目标是将引导框架作为理解在线自我改进的分析透镜。这一视角将自我演化智能体的研究从聚合成功率推进到更具原则性的研究, 即智能体如何在现实任务流中产生、组织、迁移和应用可复用的经验。

局限性

本文的研究聚焦于基于文本的工具使用、网页、命令行和软件工程环境中的单智能体大语言模型系统。我们未评估具身智能体或多智能体系统，在这些系统中，引导框架可能涉及额外的交互、协调和环境反馈形式。此外，我们以自然语言指导来实例化引导框架；其他引导框架格式，如可执行代码技能或结构化程序，不在本研究范围内。

参考文献

- Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, and 1 others. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*.
- Sandhini Agarwal, Lama Ahmad, Jason Ai, Sam Altman, Andy Applebaum, Edwin Arbus, Rahul K Arora, Yu Bai, Bowen Baker, Haiming Bao, and 1 others. 2025. gpt-oss-120b & gpt-oss-20b model card. *arXiv preprint arXiv:2508.10925*.
- Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziemis, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, and 1 others. 2025. Gepa: Reflective prompt evolution can outperform reinforcement learning. *arXiv preprint arXiv:2507.19457*.
- Anthropic. 2025. [Introducing claude opus 4.5](#).
- Anthropic. 2026a. [Introducing claude opus 4.6](#).
- Anthropic. 2026b. [Introducing claude opus 4.7](#).
- Yuanchen Bei, Tianxin Wei, Xuying Ning, Yanjun Zhao, Zhining Liu, Xiao Lin, Yada Zhu, Hendrik Hamann, Jingrui He, and Hanghang Tong. 2026. Mem-gallery: Benchmarking multimodal long-term conversational memory for mllm agents. *arXiv preprint arXiv:2601.03515*.
- Minghao Chen, Yihang Li, Yanting Yang, Shiyu Yu, Binbin Lin, and Xiaofei He. 2024. Automanual: Constructing instruction manuals by llm agents via interactive environmental learning. *Advances in Neural Information Processing Systems*, 37:589–631.
- Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. 2025. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*.
- Shihan Dou, Ming Zhang, Zhangyue Yin, Chenhao Huang, Yujiong Shen, Junzhe Wang, Jiayi Chen, Yuchen Ni, Junjie Ye, Cheng Zhang, and 1 others. 2026. Cl-bench: A benchmark for context learning. *arXiv preprint arXiv:2602.03587*.
- Lutfi Eren Erdogan, Nicholas Lee, Sehoon Kim, Suhong Moon, Hiroki Furuta, Gopala Anumanchipalli, Kurt Keutzer, and Amir Gholami. 2025. Plan-and-act: Improving planning of agents for long-horizon tasks. *arXiv preprint arXiv:2503.09572*.
- Wei-Chieh Huang, Weizhi Zhang, Yueqing Liang, Yuanchen Bei, Yankai Chen, Tao Feng, Xinyu Pan, Zhen Tan, Yu Wang, Tianxin Wei, and 1 others. 2026. Rethinking memory mechanisms of foundation agents in the second half. *arXiv preprint arXiv:2602.06052*.
- Guanyu Jiang, Zhaochen Su, Xiaoye Qu, and Yi R Fung. 2026. Xskill: Continual learning from experience and skills in multimodal agents. *arXiv preprint arXiv:2603.12056*.
- Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. 2024. [SWE-bench: Can language models resolve real-world github issues?](#) In *The Twelfth International Conference on Learning Representations*.
- Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. 2026. Meta-harness: End-to-end optimization of model harnesses. *arXiv preprint arXiv:2603.28052*.
- Minhua Lin, Hanqing Lu, Zhan Shi, Bing He, Rui Mao, Zhiwei Zhang, Zongyu Wu, Xianfeng Tang, Hui Liu, Zhenwei Dai, and 1 others. 2026a. Position: Agentic evolution is the path to evolving llms. *arXiv preprint arXiv:2602.00359*.
- Minhua Lin, Juncheng Wu, Zijun Wang, Zhan Shi, Yisi Sang, Bing He, Zewen Liu, Tianxin Wei, Zongyu Wu, Zhiwei Zhang, and 1 others. 2026b. Harness updating is not harness benefit: Disentangling evolution capabilities in self-evolving llm agents. *arXiv preprint arXiv:2605.30621*.
- Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, and 1 others. 2024. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*.
- Jiaqi Liu, Zipeng Ling, Shi Qiu, Yanqing Liu, Siwei Han, Peng Xia, Haoqin Tu, Zeyu Zheng, Cihang Xie, Charles Fleming, and 1 others. 2026a. Omnimem: Autoresearch-guided discovery of life-long multimodal agent memory. *arXiv preprint arXiv:2604.01007*.
- Zewen Liu, Zhan Shi, Yisi Sang, Bing He, Minhua Lin, Tianxin Wei, Dakuo Wang, Benoît Dumoulin, Wei Jin, and Hanqing Lu. 2026b. Adaptive auto-harness: Sustained self-improvement for agentic system deployment on open-ended task streams. *arXiv preprint arXiv:2606.01770*.
- Xinghua Lou, Miguel Lázaro-Gredilla, Antoine Dedieu, Carter Wendelken, Wolfgang Lehrach, and Kevin P Murphy. 2026. Autoharness: improving llm agents

- by automatically synthesizing a code harness. *arXiv preprint arXiv:2603.03329*.
- Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, and 1 others. 2023. Self-refine: Iterative refinement with self-feedback. *Advances in neural information processing systems*, 36:46534–46594.
- Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, E. Kelly Buchanan, Junhong Shen, Guanghao Ye, Haowei Lin, Jason Poulos, Maoyu Wang, Marianna Nezhurina, Jernia Jitsev, Di Lu, Orfeas Menis Mastromichalakis, and 66 others. 2026. [Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces](#). *Preprint*, arXiv:2601.11868.
- Jingwei Ni, Yihao Liu, Xinpeng Liu, Yutao Sun, Mengyu Zhou, Pengyu Cheng, Dexin Wang, Erchao Zhao, Xiaoxi Jiang, and Guanjun Jiang. 2026. Trace2skill: Distill trajectory-local lessons into transferable agent skills. *arXiv preprint arXiv:2603.25158*.
- Xuying Ning, Katherine Tieu, Dongqi Fu, Tianxin Wei, Zihao Li, Yuanchen Bei, Jiaru Zou, Mengting Ai, Zhining Liu, Ting-Wei Li, and 1 others. 2026. Code as agent harness. *arXiv preprint arXiv:2605.18747*.
- Siru Ouyang, Jun Yan, Yanfei Chen, Rujun Han, Zifeng Wang, Bhavana Dalvi Mishra, Rui Meng, Chunliang Li, Yizhu Jiao, Kaiwen Zha, and 1 others. 2026. Skillos: Learning skill curation for self-evolving agents. *arXiv preprint arXiv:2605.06614*.
- Siru Ouyang, Jun Yan, I Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, Long T Le, Samira Daruki, Xiangru Tang, and 1 others. 2025. Reasoningbank: Scaling agent self-evolving with reasoning memory. *arXiv preprint arXiv:2509.25140*.
- Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, and 1 others. 2024. Toolllm: Facilitating large language models to master 16000+ real-world apis. In *International Conference on Learning Representations*, volume 2024, pages 9695–9717.
- Libin Qiu, Zhirong Gao, Junfu Chen, Yuhang Ye, Weizhi Huang, Xiaobo Xue, Wenkai Qiu, and Shuo Tang. 2026. Autorefine: From trajectories to reusable expertise for continual llm agent refinement. *arXiv preprint arXiv:2601.22758*.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems*, 36:8634–8652.
- Weiwei Sun, Miao Lu, Zhan Ling, Kang Liu, Xuesong Yao, Yiming Yang, and Jiecao Chen. 2025. Scaling long-horizon llm agent via context-folding. *arXiv preprint arXiv:2510.11967*.
- Mirac Suzgun, Mert Yuksekgonul, Federico Bianchi, Dan Jurafsky, and James Zou. 2025. Dynamic cheat-sheet: Test-time learning with adaptive memory. *arXiv preprint arXiv:2504.07952*.
- Gemini Team, Rohan Anil, Sebastian Borgeaud, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, Katie Millican, and 1 others. 2023. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*.
- Kimi Team, Tongtong Bai, Yifan Bai, Yiping Bao, SH Cai, Yuan Cao, Y Charles, HS Che, Cheng Chen, Guanduo Chen, and 1 others. 2026. Kimi k2. 5: Visual agentic intelligence. *arXiv preprint arXiv:2602.02276*.
- Chenxi Wang, Zhuoyun Yu, Xin Xie, Wuguannan Yao, Runnan Fang, Shuofei Qiao, Kexin Cao, Guozhou Zheng, Xiang Qi, Peng Zhang, and 1 others. 2026. Skillx: Automatically constructing skill knowledge bases for agents. *arXiv preprint arXiv:2604.04804*.
- Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2023. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*.
- Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. 2024. Agent workflow memory. *arXiv preprint arXiv:2409.07429*.
- Tianxin Wei, Noveen Sachdeva, Benjamin Coleman, Zhankui He, Yuanchen Bei, Xuying Ning, Mengting Ai, Yunzhe Li, Jingrui He, Ed H Chi, and 1 others. 2025. Evo-memory: Benchmarking llm agent test-time learning with self-evolving memory. *arXiv preprint arXiv:2511.20857*.
- Peng Xia, Jianwen Chen, Hanyang Wang, Jiaqi Liu, Kaide Zeng, Yu Wang, Siwei Han, Yiyang Zhou, Xujiang Zhao, Haifeng Chen, and 1 others. 2026. Skillrl: Evolving agents via recursive skill-augmented reinforcement learning. *arXiv preprint arXiv:2602.08234*.
- Wujiang Xu, Kai Mei, Hang Gao, Juntao Tan, Zujie Liang, and Yongfeng Zhang. 2025. A-mem: Agentic memory for llm agents. *arXiv preprint arXiv:2502.12110*.
- Yutao Yang, Junsong Li, Qianjun Pan, Bihao Zhan, Yuxuan Cai, Lin Du, Jie Zhou, Kai Chen, Qin Chen, Xin Li, and 1 others. 2026. Autoskill: Experience-driven lifelong learning via skill self-evolution. *arXiv preprint arXiv:2603.01145*.
- Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2024. τ -bench: A benchmark for tool-agent-user interaction in real-world domains, 2024. URL <https://arxiv.org/abs/2406.12045>, 1(3).

Hongli Yu, Tinghong Chen, Jiangtao Feng, Jiangjie Chen, Weinan Dai, Qiying Yu, Ya-Qin Zhang, Wei-Ying Ma, Jingjing Liu, Mingxuan Wang, and 1 others. 2025. Memagent: Reshaping long-context llm with multi-conv rl-based memory agent. *arXiv preprint arXiv:2507.02259*.

Haozhen Zhang, Quanyu Long, Jianzhu Bao, Tao Feng, Weizhi Zhang, Haodong Yue, and Wenya Wang. 2026a. Memskill: Learning and evolving memory skills for self-evolving agents. *arXiv preprint arXiv:2602.02474*.

Kangning Zhang, Shuai Shao, Qingyao Li, Jianghao Lin, Lingyue Fu, Shijian Wang, Wenxiang Jiao, Yuan Lu, Weiwen Liu, Weinan Zhang, and 1 others. 2026b. Mmskills: Towards multimodal skills for general visual agents. *arXiv preprint arXiv:2605.13527*.

Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, and 1 others. 2025. Agentic context engineering: Evolving contexts for self-improving language models. *arXiv preprint arXiv:2510.04618*.

Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. 2024. Expel: Llm agents are experiential learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 19632–19642.

Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. 2024. Memorybank: Enhancing large language models with long-term memory. In *Proceedings of the AAAI conference on artificial intelligence*, volume 38, pages 19724–19731.

Shuyan Zhou. 2026. [Webarena-infinity: Generating browser environments with verifiable tasks at scale](#). *shuyanzhou.com*.

Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, and 1 others. 2024. Webarena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations*, volume 2024, pages 15585–15606.

附录

A 基准详情

表 5 总结了基准设置。我们在所有基准上使用相同的在线协议：任务以流的形式处理，求解器使用当前工具链，进化器在观察到任务结果后更新工具链。

表 5：基准统计与评估信号。

Benchmark	#Tasks	Domain	Evaluation signal
WebArena Inf.	80	Web navigation	State verifier
Terminal-Bench	89	CLI / scripting	Docker verifier
SWE-bench Lite	300	Software eng.	Unit tests
CL-Bench	1899	Adaptive reasoning	Rubric judge
TAU-Bench	165	Tool use	State verifier

WebArena Infinity。我们使用来自以电子邮件为中心的“超人通用”设置及相关网络应用中的 80 个困难任务。任务需要有状态的浏览器交互，例如创建标签、更改设置、使用电子邮件过滤器或协调日历/电子邮件信息。程序化验证器检查最终应用状态。该基准有助于研究技能是否捕获了用户界面过程。

Terminal-Bench v2。我们评估了 89 个任务，涵盖软件工程、数据处理、科学计算、安全、系统管理及相关命令行领域。求解器在隔离的 shell 环境中工作，并在任务特定的 Docker 镜像内提交最终工件或答案。最终成功由基准的可执行验证器确定，通常是在容器内运行的隐藏或挂载测试脚本。

SWE-bench Lite。我们使用来自 12 个 Python 仓库的 300 个真实 GitHub 问题。求解器编辑代码，最终补丁由仓库测试评估。

CL-Bench。CL-Bench 包含 1899 个自适应推理任务，涵盖领域知识推理、经验发现与仿真、程序性任务执行以及规则系统应用。每个任务均依据评分标准进行评定。

TAU-Bench (TAU-Bench)。TAU-Bench 包含 165 个跨航空和零售领域的客户服务工具使用任务。评估检查最终数据库状态。

B 测试工件示例

本节展示具有代表性的学习工件，而不逐一列举学习清单。对于

在任务级比较中，我们将无进化运行与进化运行之间的相同任务标识符对齐。仅当基线运行失败而进化运行成功时，才将该案例称为直接改进。这不是单一技能的因果消融：多个技能可能被同时注入，且某些基准测试并非在每个结果文件中都记录注入的技能。因此，示例采用结构化的失败/成功格式：任务、验证器信号、轨迹摘要、所学技能内容以及简短要点。

WebArena: verified setting

- **Task:** task_h10; create “Legal” and disable “Support Ticket”.
- **Baseline**
验证器发现“支持工单”仍处于启用状态，尽管轨迹声称两个子目标均已完成。
- **Evolved**
验证器通过；轨迹返回自动标签列表并检查了现有标签状态。
- **习得技能:** 关闭现有自动标签；不删除；独立验证创建与禁用。

核心要点。该框架习得了一个具体的界面区分：创建新标签与禁用现有标签是独立操作。

WebArena: grounded recipients

- **Task**
task_h65; 回复 Kevin Zhao，抄送首席技术官与工程副总裁。
- **Baseline**
发送至看似合理的姓名但域名错误（acme.com 而非 acmecorp.com）。
- **Evolved**
验证器通过，抄送中包含所需的角色派生收件人。
- **习得技能:** 优先使用权威角色来源，并在各应用间保留准确的电子邮件地址。

Core point.

该框架学习的是身份锚定，而非仅仅撰写电子邮件。

Terminal-Bench: executable recovery

- **任务:** 自适应拒绝采样器；通过针对 R 采样器的 Docker 验证器。
- **Baseline**
在重复运行相同的 R 测试命令后，正式测试仍然失败。
- **Evolved**
在诊断出超时、缺失依赖、正式测试以及排序向量失败后通过。
- **习得技能:** 在反复出现类似失败后，停止，缩小案例，构建最小复现器，然后再逐步增加复杂度。

核心要点。该框架学习了一种针对可执行反馈循环的恢复策略。

SWE-bench: test-grounded repair

- **Task:** django__django-11133; fix Django response handling without regressing existing behavior.
- **Baseline**
在源代码搜索和失败的测试尝试后超时。
- **Evolved:** passed FAIL_TO_PASS: 1/1 and PASS_TO_PASS: 64/64.
- **习得技能:** 阅读完整的失败测试断言，匹配确切的预期行为，然后在提交前运行针对性测试。

核心要点。该框架学会了将补丁锚定在可执行的测试预期中。

TAU-Bench: complete tool execution

- **Task**
retail_task_5; 从一个订单中交换多个商品。
- **Baseline**
数据库验证器失败，与部分执行或错误工具
- **Evolved** 执行的情况一致。
检索订单并应用所有请求的商品更改后，验证器通过。
- **习得技能:** 枚举每个请求的操作，根据订单状态选择工具，并验证响应是否覆盖完整列表。

核心要点。 The harness learned complete tool-call execution.

C 哪些任务更容易改进？

最具信息量的模式不是失败到通过案例的总数，而是框架演化后发生转变的任务类型。表 6 按可观察的失败模式对匹配的失败到通过案例进行分组，并给出输出中运行的具有代表性的任务标识符。

D 框架实际学习的内容

图 7 展示了选取的具有匹配的失败-通过支持的真实技能片段。证据行报告了运行中相关的转向家族，而规则行给出了核心学习到的行为。

E 提示模板

这些提示在所有实验中遵循相同的上下文到框架的编译逻辑：求解器反思执行上下文以提出候选经验，进化器根据现有框架筛选这些候选经验。下面，我们提供代表性的提示模板，以捕捉这一共同的提出与筛选过程。

E.1 求解器端提出提示

Proposal prompt, compressed

Inputs

评估结果、验证器细节或评分标准反馈、轨迹信号、压缩轨迹以及相关的现有技能。

分析执行过程。对于每个不同的问题，识别出错误之处以及具体缺失的动作、命令、工具调用、导航步骤或领域规则。

提出一个可复用的技能。选择一个宽泛的主题，决定是新建、增强还是无操作，撰写触发式描述，并提供包含技巧和注意事项的简短要点内容。

积极过滤。跳过通用建议、基础工具用法、完全相同的任务复现，以及无法帮助处理未见任务的技能。

E.2 进化端策展人提示词

Curator prompt, compressed

Inputs

主题、当前技能库、预算以及最新一批提案。对于每个提案，选择接受、合并或跳过。优先合并而非重复，尊重预算，要求清晰的触发描述，保持内容简短且可操作，并应用通用性测试，例如对多个未见任务是否有用。

E.3 通用技能策展人

General skill curator, compressed

分析跨任务的重复模式，而非单一主题内的模式。仅当模式在多个上下文中出现时，才创建或更新通用技能。通用技能必须避免特定上下文的引用，并编码能够指导跨任务的规划、验证、恢复或工具使用的流程。

F 实验细节

所有实验均使用相同的开源流水线，并配有特定基准的适配器。WebArena 使用具有持久应用状态的浏览器工作进程。Terminal-Bench 使用任务特定的 Docker 镜像和可执行验证器。SWE-bench 使用隔离的仓库环境和单元测试评估。TAU-Bench 使用官方工具使用环境和验证器。CL-Bench 使用基于评分标准的评判流水线。该框架以 Markdown 技能文件形式存储，并带有轻量级 YAML 元数据，使学习到的工件可检查且易于在不同运行之间迁移。每个条目包含一个触发器，描述何时检索该条目；一条简短的可操作规则或程序；可选的证据，将其与先前的执行关联起来；以及一个作用域，指示其意图是作为跨任务指导还是任务类型

表 6: 来自匹配的基线失败/进化通过案例的改进热点。

Benchmark	Easier-to-improve tasks	Representative cases	Why skills help
WebArena	Stateful UI tasks with exact recipients, labels, settings, and mail state.	h10: disable an existing auto label; h65: preserve corporate email domains; h73: archive all qualifying read-important emails.	The verifier checks final app state, so procedural memory about where to click must be coupled with exact state confirmation.
Terminal-Bench	Tasks with executable feedback loops and recoverable failures.	adaptive-rejection-sampler, dna-insert, build-pmars, regex-chess.	The solver can observe Docker-test failures; process skills help it pivot, reduce scope, and verify artifacts before submission.
SWE-bench Lite	Mature repositories with stable conventions, especially Django and Sympy.	django__django-11133, django__django-17087, sympy__sympy-19254.	The issue often hides exact expectations in tests or framework internals; topic skills preserve these conventions across later tasks.
TAU-Bench	Retail exchanges/returns and airline reservation edits with multiple constraints.	retail_task_5, retail_task_91, airline_task_13.	The state verifier rewards complete tool-call execution. Skills prevent partial updates, wrong mutation tools, and guessed item or flight identifiers.
CL-Bench	Rule-application, procedural, and domain-knowledge contexts with local rubrics.	Context categories with many turnarounds include Rule System Application, Procedural Task Execution, and Domain Knowledge Reasoning.	Context skills behave like small local manuals: exact extraction rules, role boundaries, output schemas, and calculation conventions.

指导。对于框架选择，我们在所有实验中使用 Claude Sonnet 4.5，在任务执行前从当前框架中检索相关技能。

除非另有说明，我们使用批大小为 16。我们将通用技能的最大数量以及每个任务类型主题下的技能最大数量设置为 5。此预算鼓励进化器合并重叠的指导，并避免积累过于详细或任务特定的记录。

G 潜在风险

EVO-HARNESS 进化技能库，这些技能库作为程序性指导注入到智能体提示中。学习到的技能旨在捕获任务解决模式，例如导航流程、工具使用程序、验证步骤和恢复策略。在我们的实验中，所有基准均在沙盒环境中运行，无法访问真实用户数据、外部账户或生产系统。因此，所评估的设置除了标准基准执行之外，带来的实际风险极小。

H 使用或创建科学工件

本文工作使用了五个已建立的公开基准：WebArena-Infinity、TerminalBench2、SWE-bench Lite、CL-Bench 和 TAU-Bench。我们未对这些基准本身进行任何修改。我们通过其标准的 AWS Bedrock 应用程序编程接口使用基础模型（Claude 及其他模型）。我们将在论文发表后发布 EVO-HARNESS 流水线代码及所有演化后的技能库。

H.1 引用工件创建者

所有基准和模型均在正文中进行了适当引用。

H.2 讨论工件许可证

本文使用的所有基准资源均在其各自的研究或开源许可证下公开可用。具体而言，WebArena Infinity 和 TAU-Bench 在 MIT 许可证下发布，WebArena 在 Apache-2.0 许可证下发布，CL-Bench 在自定义的仅限评估许可证下发布。我们的流水线代码将在 MIT 许可证下发布。



图 7：跨基准的代表性学习到的约束摘录。每张卡片将反复出现的失败-通过模式与相应的演化指导配对。百分比总结了匹配的基线失败/演化通过案例，并为示例提供背景，而非建立单一技能的因果关系。

H.3 工件使用符合预期

用途

所有基准均用于其评估智能体能力的预期目的。基础模型通过其标准应用程序编程接口使用，并符合其服务条款。

H.4 数据包含个人身份信息

或冒犯性内容

基准使用合成或脱敏场景。SWE-bench 使用公开的 GitHub 问题。据我们所知，其中不包含个人身份信息或冒犯性内容。

I 计算实验

I.1 计算预算

完整的实验套件计算成本高昂，因为它涵盖了多个基准、求解器模型、进化设置和消融配置。在所有主要实验和诊断运行中，总的应用程序编程接口成本约为 10 万美元。对于单个基准配置，我们通常运行 16 个并行工作进程。根据基准复杂度、任务超时和环境开销，一次完整运行大约需要 2 到 8 小时的墙钟时间。网络和工具使用基准通常更快，而

软件工程和命令行任务往往需要更长时间，因为它们需要仓库设置、Docker 执行、测试运行或验证器调用。

I.2 实验设置与超参数

关键超参数在附录 F 中描述。我们使用固定的随机种子（42）进行任务洗牌，以确保可重复性。

I.3 描述性统计

我们报告所有基准的通过率（%）作为主要度量。为了统计可靠性，我们报告每个基准的完整任务集结果，而不是子采样。性能是三次运行的平均值。

I.4 软件包参数

本项目主要使用亚马逊云科技基岩（AWS Bedrock）应用程序编程接口进行基础模型推理，并配合标准亚马逊云科技基础设施服务，如弹性计算云（EC2）、简单存储服务（S3）、弹性块存储/文件存储（EBS/FSx）、云监控（CloudWatch）、身份与访问管理（IAM）和虚拟私有云（VPC）。软件环境使用 Python 3.10+、boto3 1.34+、botocore 1.34+、PyTorch 2.0+、CUDA 12.0+、Docker 和 Ubuntu 24.04。

J 研究中或写作中的人工智能助手

本文以大型语言模型（LLMs）为研究对象，并在实验推理中使用冻结的大型语言模型骨干网络。

这些模型不用于创建新的基准标签或伪造实验结果。报告中的数值结果来自实验代码，并由作者核实。人工智能助手工具用于润色稿件。它们不用于决定科学主张、选择报告结果或替代作者验证。